NetHub: o que revisar e melhorar

Plano compartilhável em commits pequenos e ordenados por dependência. Escopo: documentação, análise e processo de entrega; sem implementar código.

Como interpretar o relatório. O arquivo anexado avalia a Entrega 1 e registra que não havia código naquele momento. O repositório descreve hoje uma ferramenta de infraestrutura; o relatório cobra também gerenciamento de usuários. Confirme essa diferença antes de tratar requisitos de usuário como escopo vigente.

Sequência recomendada

Os blocos 00 e 01 estabelecem a linha de base. Os próximos dependem dessa decisão. Se a equipe confirmar gerenciamento de usuários como escopo das sprints, siga os blocos 02 a 07; depois conclua o processo e a entrega.

00 · Confirmar o escopo com a equipe

Commit sugerido: `docs: registrar escopo e critérios confirmados`

Objetivo: Resolver a divergência entre o relatório da disciplina e a documentação vigente.

• Registrar se gerenciamento de usuários é obrigatório nas Sprints 1 e 2.

• Confirmar nome, perfis de usuário, limites do produto e relação entre a ferramenta de infraestrutura e o sistema de usuários.

• Anotar decisão, validadores e fonte (modelo da disciplina, professor ou equipe); não inventar decisões em nome do grupo.

Condição: Não iniciar os blocos específicos de usuários sem essa decisão.

01 · Atualizar a linha de base documental

Commit sugerido: `docs: alinhar visão, entregas e navegação`

Objetivo: Alinhar escopo, visão, checklist e documentos atuais.

• Revisar docs/01-visao.md, docs/00-entregas.md e README.md após a decisão do bloco 00.

• Preservar anexos originais e indicar o que é legado, vigente ou pendente.

• Alinhar glossário e elicitação; registrar apenas evidências realmente coletadas.

Ramo A · se gerenciamento de usuários for confirmado

02 · Especificar requisitos de usuário

Commit sugerido: `docs: especificar requisitos de gerenciamento de usuários`

Objetivo: Cobrir o núcleo de usuários apontado como ausente nos RFs.

• Criar ou revisar RFs de cadastro, listagem e autenticação, com IDs estáveis, prioridade e ao menos um cenário por requisito.

• Definir regras de login e senha conforme o enunciado oficial; registrar fontes e limites.

• Rastrear requisitos, casos de uso e critérios da entrega.

Condição: Confirmar se a política AWS IAM e as validações descritas no relatório são exatamente as exigidas.

03 · Revisar requisitos não funcionais

Commit sugerido: `docs: tornar critérios não funcionais verificáveis`

Objetivo: Trocar termos vagos por critérios observáveis e registrar cobertura.

• Revisar “tempo aceitável”, “de forma graciosa” e “íntegra”; definir medidas, limites e condições com a equipe.

• Manter critérios mensuráveis ainda válidos; marcar os que não se aplicam.

• Vincular cenários e métodos de verificação.

04 · Completar casos de uso e diagrama

Commit sugerido: `docs: modelar casos de uso de usuários`

Objetivo: Modelar a interação pedida para gerenciamento de usuários.

• Criar diagrama com pelo menos dois tipos de usuário se o critério for confirmado.

• Revisar fluxos alternativos e exceções nos casos textuais.

• Atualizar docs/04-casos-de-uso/README.md e rastreabilidade.

05 · Completar o modelo de análise

Commit sugerido: `docs: completar diagrama de análise`

Objetivo: Entregar classes de fronteira, controle e entidade coerentes com os casos de uso.

• Revisar docs/05-analise.md e o diagrama atual sem antecipar implementação.

• Representar exceções e dois mecanismos de persistência se fizerem parte do escopo.

• Validar responsabilidades e relacionamentos com os requisitos.

06 · Documentar arquitetura e persistência

Commit sugerido: `docs: registrar alternativas de persistência`

Objetivo: Documentar mecanismos de persistência e seleção no início da execução se exigidos.

• Registrar alternativas, critérios, responsabilidades por camada e falhas de E/S.

• Atualizar docs/07-arquitetura/ e ADRs; se não houver decisão, documentar opções e pendência.

• Conferir consistência entre análise, arquitetura e requisitos.

07 · Organizar pacotes e projeto

Commit sugerido: `docs: documentar pacotes e projeto`

Objetivo: Preencher divisão de pacotes/camadas e artefatos de projeto.

• Completar docs/08-projeto.md com responsabilidades e dependências.

• Reconciliar com docs/05-analise.md e docs/07-arquitetura/.

• Separar proposta conceitual de decisão aprovada.

Processo de entrega e fechamento

08 · Adotar fluxo de Git e contribuição

Commit sugerido: `docs: definir fluxo de contribuição da equipe`

Objetivo: Responder às observações sobre mensagens de commit e fluxo de revisão.

• Seguir CONTRIBUTING.md e usar commits pequenos com mensagens claras.

• Combinar branches, PRs e integração em develop/main conforme a disciplina.

• Criar tags/releases se fizerem parte do fluxo acordado; não reescrever histórico.

09 · Fechar checklist e preparar entrega

Commit sugerido: `docs: fechar checklist da entrega 2`

Objetivo: Deixar evidências e pendências fáceis de conferir.

• Atualizar docs/00-entregas.md com responsáveis, status e datas oficiais confirmadas.

• Conferir IDs, links, diagramas, fontes e consistência.

• Registrar itens fora do escopo ou dependentes de validação.

Resumo: escopo → linha de base → requisitos RF → RNF → casos de uso → análise → arquitetura/persistência → pacotes/projeto → fluxo Git → checklist. Cada bloco pode ser um commit; combine blocos apenas quando produzirem uma alteração documental única e coerente. Este plano não implementa funcionalidades.